

Question 1: Library Book Request Management System

A university library maintains a dynamic list of book requests submitted by students. Each request contains the following details:

- Request ID
- Student Roll Number
- Book ISBN
- Request Date

The system must support the following operations:

1. Add a new request to the end of the list.
2. Remove a request once the book has been issued.
3. Display all pending requests in the order they were received.
4. Count the total number of pending requests.

The library processes requests strictly in the order of arrival, and once a request is fulfilled, it will never be revisited.

Task

Design and implement an efficient data structure in **C** to support the above operations. Write functions for each operation and demonstrate their working using a menu-driven program.

Revised Question 2: Text Editor Undo–Redo System

A text editor maintains a history of user actions to support undo and redo functionality. Each action contains:

- Action ID
- Action Type (Insert / Delete / Replace)
- Timestamp

- Description of the change

The editor allows the following operations:

1. Perform a new action and add it to the history.
2. Undo the most recent action.
3. Redo an undone action.
4. Delete a specific action from the history.
5. Display the complete action history from the first action to the most recent one.

Once a new action is performed after an undo operation, all actions that were undone must be permanently discarded.

Navigation through the action history must support moving both backward and forward efficiently without restarting from the beginning.

Task

Select an appropriate dynamic data structure and implement the above functionality in **C**. Your implementation should correctly handle edge cases such as undoing the first action, redoing the last action, and deleting the current action.

Question 3: CPU Task Scheduling in an Embedded System

An embedded operating system manages a set of real-time tasks. Each task contains:

- Task ID
- Priority
- Execution Time

The scheduler repeatedly executes tasks one by one in a fixed order. After the last task is executed, the scheduler immediately returns to the first task and continues execution until the system is shut down.

The system must support:

1. Adding a new task at runtime.

2. Removing a completed task.
3. Displaying all active tasks starting from the currently executing task.
4. Simulating the execution of n task cycles.

The scheduler should never encounter a NULL task reference during execution.

Task

Design and implement an appropriate data structure in **C** to simulate the task scheduler. Provide a menu-driven program to demonstrate all required operations.